In [1]:

```python
# Project : RetailPulse - Comprehensive Retail Data Analysis.

# Load and clean sales data with Pandas.
# Conduct EDA to understand data distribution,and trends.
# Visualize trends with Matplotlib/Seaborn.

# Disclaimer: The dataset used in this analysis is simulated (dummy data) and does not contain real customer information.
# It is intended solely for practice and demonstration of data analysis techniques.

# Install required packages.

!pip install pandas numpy matplotlib seaborn
!python -m pip install --upgrade pip
!pip install openpyxl
!pip install --upgrade pip

# Imports.

import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
Requirement already satisfied: pandas in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (2.2.0)
Requirement already satisfied: numpy in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (1.26.4)
Requirement already satisfied: matplotlib in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (3.8.2)
Requirement already satisfied: seaborn in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (4.48.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (1.4.5)
Requirement already satisfied: packaging>=20.0 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (23.2)
Requirement already satisfied: pillow>=8 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from matplotlib) (3.1.1)
Requirement already satisfied: six>=1.5 in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Requirement already satisfied: pip in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (26.1)
Requirement already satisfied: openpyxl in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: pip in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (26.1)
```

In [2]:

```python
# Read dataset and state it's shape.

EXCEL_PATH = "Sales Data.xlsx"
df = pd.read_excel(EXCEL_PATH, engine="openpyxl")

print("Initial Shape:", df.shape)
df.head()
```

```
Initial Shape: (10500, 20)
```

Out[2]:

	UserID	CustomerName	ProductID	Gender	AgeGroup	Age	MaritalStatus	State	Occupation	ProductCategory	Quantity	Amount	Discount	FinalAmount	OrderStatus	PaymentMode	OrderDate	DeliveryDate	Column1	Column2
0	5119	Pahal Sethi	P5924	Male	18-25	24.0	Married	State11	Technician	Clothing	5	929.03	NaN	929.03	Cancelled	Wallet	08-23-2025	NaN	NaN	NaN
1	8932	Yasti Patla	P1391	Female	NaN	NaN	Married	State11	Business	Music	5	6778.53	684.27	6094.26	Returned	NaN	2025-09-04 00:00:00	NaN	NaN	NaN
2	8516	Aryan Dugal	P3250	Male	NaN	NaN	???	State19	Salesperson	Healthcare	2	9978.75	1720.35	8258.40	Pending	Net Banking	2025-08-02 00:00:00	NaN	NaN	NaN
3	7283	Madhavi Butala	P2750	Male	26-35	26.0	Single	State6	Manager	Baby Products	5	8589.02	581.49	8007.53	Delivered	Credit Card	06-25-2025	2025-07-05 00:00:00	NaN	NaN
4	7624	Ishaan Bava	P5426	Female	36-45	36.0	Married	State17	Lawyer	Stationery	5	9565.05	NaN	9565.05	Returned	UPI	2025-06-16 00:00:00	NaN	NaN	NaN

In [3]:

```python
# Dataset - Column names.

df.columns.tolist()
```

Out[3]:

```
['UserID',
 'CustomerName',
 'ProductID',
 'Gender',
 'AgeGroup',
 'Age',
 'MaritalStatus',
 'State',
 'Occupation',
 'ProductCategory',
 'Quantity',
 'Amount',
 'Discount',
 'FinalAmount',
 'OrderStatus',
 'PaymentMode',
 'OrderDate',
 'DeliveryDate',
 'Column1',
 'Column2']
```

In [4]:

```python
# Top 5 records of the dataset.

df.head()
```

Out[4]:

	UserID	CustomerName	ProductID	Gender	AgeGroup	Age	MaritalStatus	State	Occupation	ProductCategory	Quantity	Amount	Discount	FinalAmount	OrderStatus	PaymentMode	OrderDate	DeliveryDate	Column1	Column2
0	5119	Pahal Sethi	P5924	Male	18-25	24.0	Married	State11	Technician	Clothing	5	929.03	NaN	929.03	Cancelled	Wallet	08-23-2025	NaN	NaN	NaN
1	8932	Yasti Patla	P1391	Female	NaN	NaN	Married	State11	Business	Music	5	6778.53	684.27	6094.26	Returned	NaN	2025-09-04 00:00:00	NaN	NaN	NaN
2	8516	Aryan Dugal	P3250	Male	NaN	NaN	???	State19	Salesperson	Healthcare	2	9978.75	1720.35	8258.40	Pending	Net Banking	2025-08-02 00:00:00	NaN	NaN	NaN
3	7283	Madhavi Butala	P2750	Male	26-35	26.0	Single	State6	Manager	Baby Products	5	8589.02	581.49	8007.53	Delivered	Credit Card	06-25-2025	2025-07-05 00:00:00	NaN	NaN
4	7624	Ishaan Bava	P5426	Female	36-45	36.0	Married	State17	Lawyer	Stationery	5	9565.05	NaN	9565.05	Returned	UPI	2025-06-16 00:00:00	NaN	NaN	NaN

In [5]:

```python
# Drop unecessary columns.

df.drop(columns=["Column1", "Column2"], inplace=True,errors="ignore")
```

In [6]:

```python
# Shape of dataset ( i.e. after dropping unecessary columns).

df.shape
```

Out[6]:

```
(10500, 18)
```

In [7]:

```python
# Converting columns into numeric in pandas.

# This code makes sure that important columns (Age, Quantity, Amount, Discount, FinalAmount) only contain numbers.
# If something is not a number, it becomes NaN.

numeric_cols = ["Age", "Quantity", "Amount", "Discount", "FinalAmount"]

df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors="coerce")

print(df[numeric_cols].dtypes)

# errors="coerce" → if a value cannot be converted (like "abc", "???", "N/A"), it is replaced with NaN (missing value).
```

```
Age            float64
Quantity         int64
Amount         float64
Discount       float64
FinalAmount    float64
dtype: object
```

In [8]:

```python
# Convert OrderDate and DeliveryDate into proper datetime.

for col in ["OrderDate", "DeliveryDate"]:
    df[col] = pd.to_datetime(df[col], errors="coerce", dayfirst=True)
    print(col, "Type:", df[col].dtype, "| Invalid to NaT:", df[col].isna().sum())


# errors="coerce" → bad dates → NaT.

# dayfirst=True → reads 06/09/2025 as 6 Sept 2025 (DD/MM/YYYY).

# dtype → shows column type after conversion.

# isna().sum() → counts invalid dates replaced by NaT.
```

```
OrderDate Type: datetime64[ns] | Invalid to NaT: 0
DeliveryDate Type: datetime64[ns] | Invalid to NaT: 7886
```

In [9]:

```python
# Handling missing values.

df.replace(["NaN", "???"], np.nan, inplace = True )
missing_before = df.isna().sum()

# Numeric columns
df["Age"] = df["Age"].fillna(df["Age"].median())
df[["Quantity", "Amount", "FinalAmount", "Discount"]] = df[["Quantity", "Amount", "FinalAmount", "Discount"]].fillna(0)

# Categorical columns
df[["AgeGroup", "PaymentMode", "Gender", "MaritalStatus", "Occupation","OrderStatus"]] = df[["AgeGroup", "PaymentMode", "Gender",
 "MaritalStatus", "Occupation","OrderStatus"]].fillna("Unknown").astype(str)

# Dates → keep NaT (no filling). Dates are time series data → filling with "Unknown" or today's date would distort results.
# NaN → missing numbers (auto), NaT → missing/invalid dates (auto), pd.NA → missing for string/categorical (auto in new pandas), "null" / "NaN" / "N/A"
# / "" / " " → appear only if written as text in dataset (not automatic). ✅


# Record missing values after cleaning
missing_after = df.isna().sum()

# Combine into a DataFrame for comparison
missing_report = pd.DataFrame({
    "Before Cleaning": missing_before,
    "After Cleaning": missing_after
})

print(missing_report)
```

```
                 Before Cleaning  After Cleaning
UserID                         0               0
CustomerName                   0               0
ProductID                      0               0
Gender                         0               0
AgeGroup                    5210               0
Age                         5210               0
MaritalStatus               3413               0
State                          0               0
Occupation                   453               0
ProductCategory                0               0
Quantity                       0               0
Amount                         0               0
Discount                    5217               0
FinalAmount                    0               0
OrderStatus                    0               0
PaymentMode                 1553               0
OrderDate                      0               0
DeliveryDate                7886            7886
```

In [10]:

```python
# Renaming of columns.

df.rename(columns={
    "UserID": "User ID",
    "CustomerName": "Customer Name",
    "ProductID": "Product ID",
    "AgeGroup": "Age Group",
    "MaritalStatus": "Marital Status",
    "ProductCategory": "Product Category",
    "FinalAmount": "Final Amount",
    "OrderStatus": "Order Status",
    "PaymentMode": "Payment Mode",
    "OrderDate": "Order Date",
    "DeliveryDate": "Delivery Date",
}, inplace=True)
```

In [11]:

```python
# Columns (after renaming) .

df.columns
```

Out[11]:

```
Index(['User ID', 'Customer Name', 'Product ID', 'Gender', 'Age Group', 'Age',
       'Marital Status', 'State', 'Occupation', 'Product Category', 'Quantity',
       'Amount', 'Discount', 'Final Amount', 'Order Status', 'Payment Mode',
       'Order Date', 'Delivery Date'],
      dtype='object')
```

In [12]:

```python
# Remove duplicates.

removed = df.duplicated().sum()
df = df.drop_duplicates().reset_index(drop=True)
print("Removed", removed, "duplicates")
```

```
Removed 500 duplicates
```

In [13]:

```python
# Derived Features.

df["DeliveryTime"] = (df["Delivery Date"] - df["Order Date"]).dt.days
df["Weekday"] = df["Order Date"].dt.day_name().fillna("Unknown")
df["Month"] = df["Order Date"].dt.month_name().fillna("Unknown")
df["Week"] = df["Order Date"].dt.isocalendar().week

print("New Shape:", df.shape)
```

```
New Shape: (10000, 22)
```

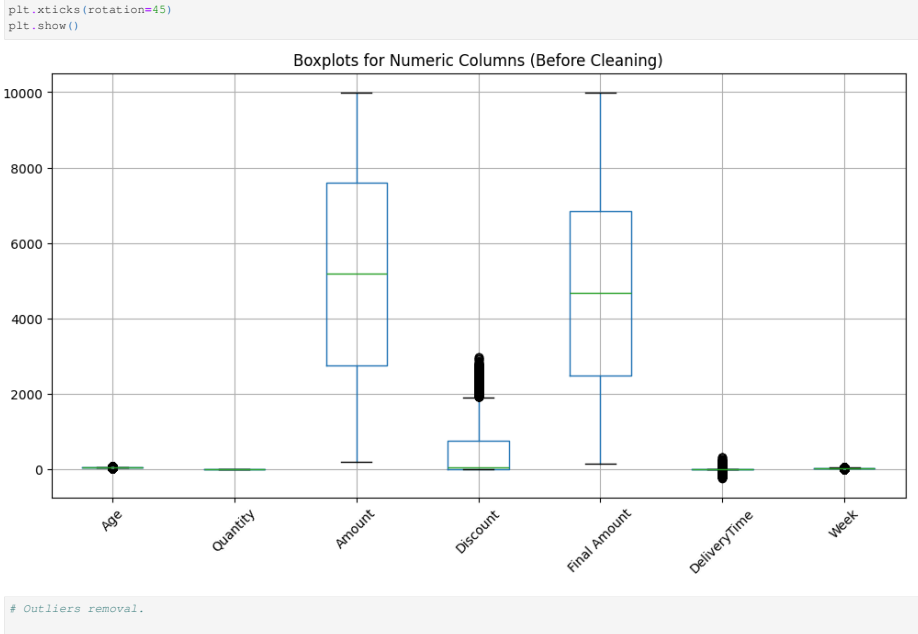
In [14]:

```python
# Summary of dataframe.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 22 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   User ID           10000 non-null  int64
 1   Customer Name     10000 non-null  object
 2   Product ID        10000 non-null  object
 3   Gender            10000 non-null  object
 4   Age Group         10000 non-null  object
 5   Age               10000 non-null  float64
 6   Marital Status    10000 non-null  object
 7   State             10000 non-null  object
 8   Occupation        10000 non-null  object
 9   Product Category  10000 non-null  object
 10  Quantity          10000 non-null  int64
 11  Amount            10000 non-null  float64
 12  Discount          10000 non-null  float64
 13  Final Amount      10000 non-null  float64
 14  Order Status      10000 non-null  object
 15  Payment Mode      10000 non-null  object
 16  Order Date        10000 non-null  datetime64[ns]
 17  Delivery Date     2480 non-null   datetime64[ns]
 18  DeliveryTime      2480 non-null   float64
 19  Weekday           10000 non-null  object
 20  Month             10000 non-null  object
 21  Week              10000 non-null  UInt32
dtypes: UInt32(1), datetime64[ns](2), float64(5), int64(2), object(12)
memory usage: 1.6+ MB
```

In [15]:

```python
# Outliers detection.

# 1) Show boxplots for all numeric columns BEFORE cleaning
numeric_cols = df.select_dtypes(include=[np.number]).columns.drop("User ID")
plt.figure(figsize=(12, 6))
df[numeric_cols].boxplot()
plt.title("Boxplots for Numeric Columns (Before Cleaning)")
plt.xticks(rotation=45)
plt.show()
```

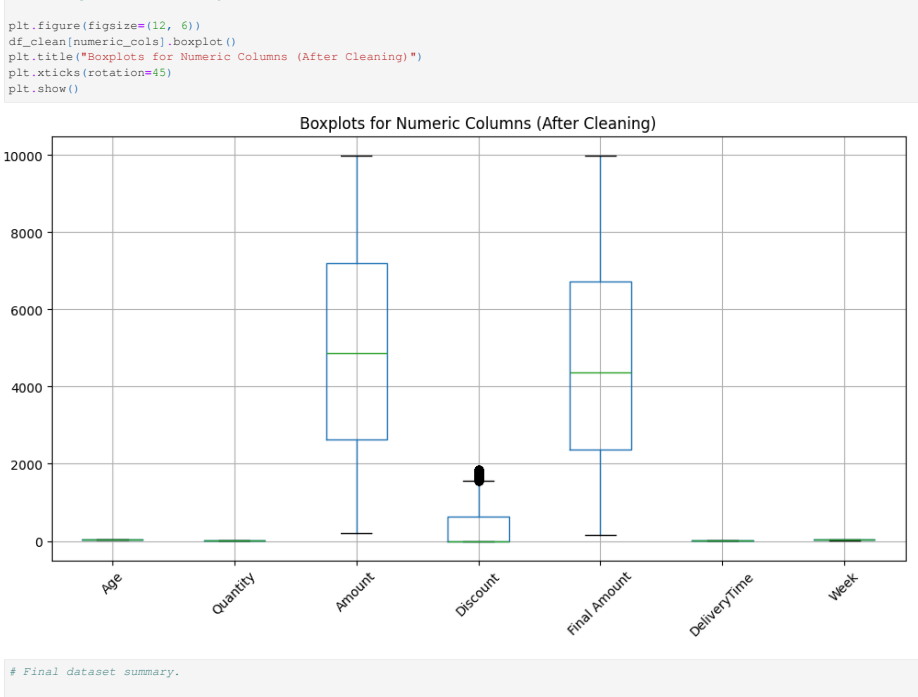


In [16]:

```python
# Outliers removal.

# 2) Outlier removal using IQR for ALL numeric columns
df_clean = df.copy()
for col in numeric_cols:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df_clean = df_clean[(df_clean[col].between(lower, upper)) | (df_clean[col].isna())]
```

In [17]:

```python
# Show boxplots AFTER cleaning.

plt.figure(figsize=(12, 6))
df_clean[numeric_cols].boxplot()
plt.title("Boxplots for Numeric Columns (After Cleaning)")
plt.xticks(rotation=45)
plt.show()
```



In [18]:

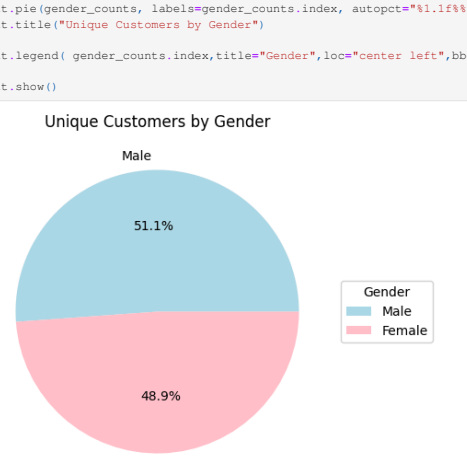
```python
# Final dataset summary.

print("Original shape:", df.shape)
print("After removing outliers:", df_clean.shape)

# Use df_clean for all visualizations.

df = df_clean.copy()
```

```
Original shape: (10000, 22)
After removing outliers: (4319, 22)
```

Data Visualization

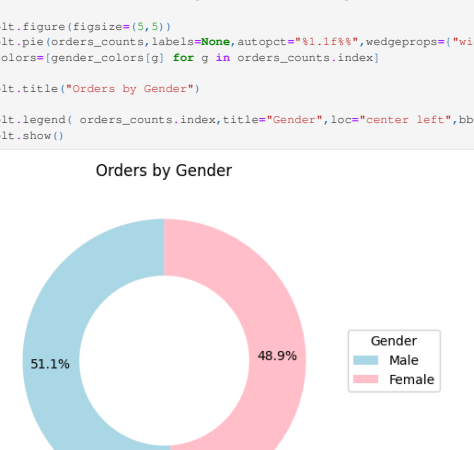
In [19]:

```python
# Create unique key.
df["CustomerKey"] = df["User ID"].astype(str) + "_" + df["Customer Name"]

# Unique customers.
unique_customers = df.drop_duplicates(subset="CustomerKey")

# 1. Gender distribution (unique customers).
gender_counts = unique_customers["Gender"].value_counts()

plt.figure(figsize=(5,5))
plt.pie(gender_counts, labels=gender_counts.index, autopct="%1.1f%%", colors=[{"Male":"lightblue","Female":"pink"}[g] for g in gender_counts.index])
plt.title("Unique Customers by Gender")

plt.legend( gender_counts.index,title="Gender",loc="center left",bbox_to_anchor=(1, 0.5) )

plt.show()
```



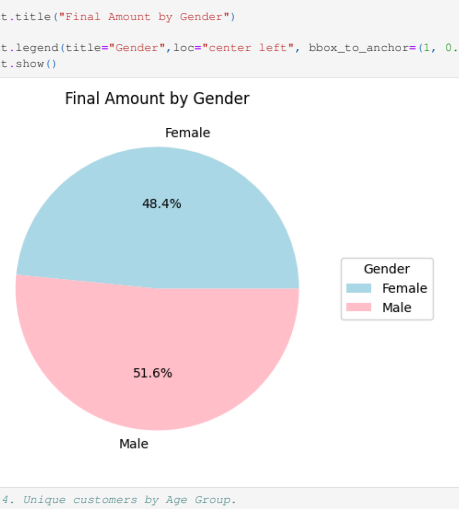
In [20]:

```python
# 2. Orders by Gender.

orders_counts = df["Gender"].value_counts()

gender_colors = {"Male": "lightblue", "Female": "pink"}

plt.figure(figsize=(5,5))
plt.pie(orders_counts,labels=None,autopct="%1.1f%%",wedgeprops={"width":0.4},startangle=90,pctdistance=0.8,
colors=[gender_colors[g] for g in orders_counts.index]
)
plt.title("Orders by Gender")

plt.legend( orders_counts.index,title="Gender",loc="center left",bbox_to_anchor=(1, 0.5))
plt.show()
```



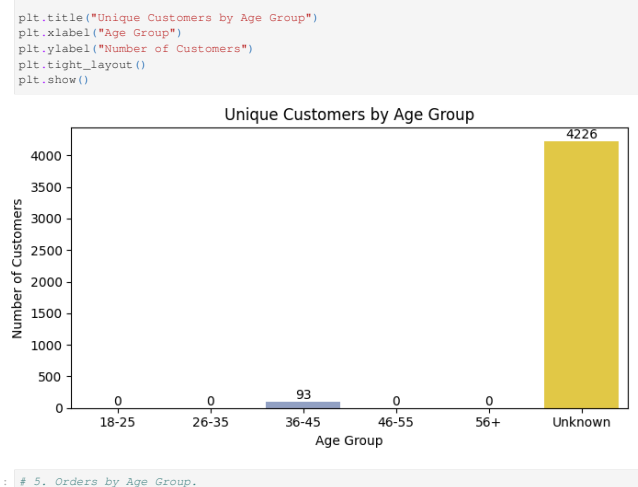
In [21]:

```python
# 3. Final amount by Gender.

final_by_gender = df.groupby("Gender")["Final Amount"].sum()

gender_colors = {"Male": "lightblue", "Female": "pink"}

plt.figure(figsize=(5,5))
plt.pie(final_by_gender, labels=final_by_gender.index, autopct="%1.1f%%",colors=[gender_colors[g] for g in orders_counts.index] )

plt.title("Final Amount by Gender")

plt.legend(title="Gender",loc="center left", bbox_to_anchor=(1, 0.5) )
plt.show()
```



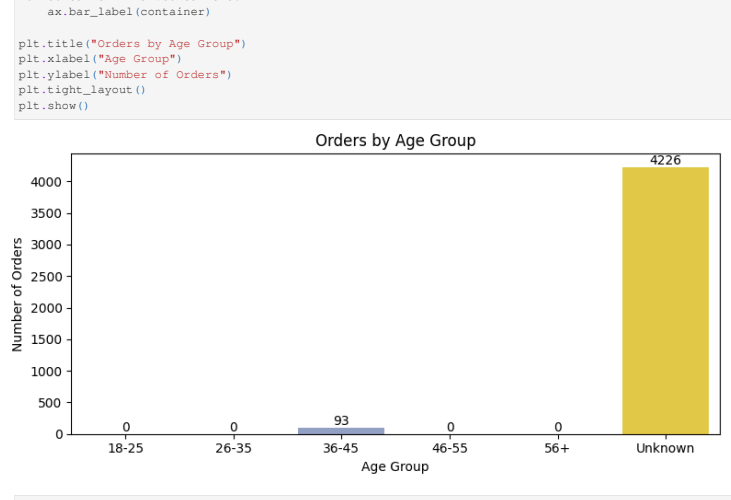
In [22]:

```python
# 4. Unique customers by Age Group.

# Create unique key.
df["CustomerKey"] = df["User ID"].astype(str) + "_" + df["Customer Name"]

# Unique customers.
unique_customers = df.drop_duplicates(subset="CustomerKey")

# Define all age group categories.
age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

age_counts = unique_customers["Age Group"].value_counts().reindex(age_order, fill_value=0)

plt.figure(figsize=(7,4))
ax = sns.barplot(x=age_counts.index, y=age_counts.values, palette="Set2")

for container in ax.containers:
    ax.bar_label(container)

plt.title("Unique Customers by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```



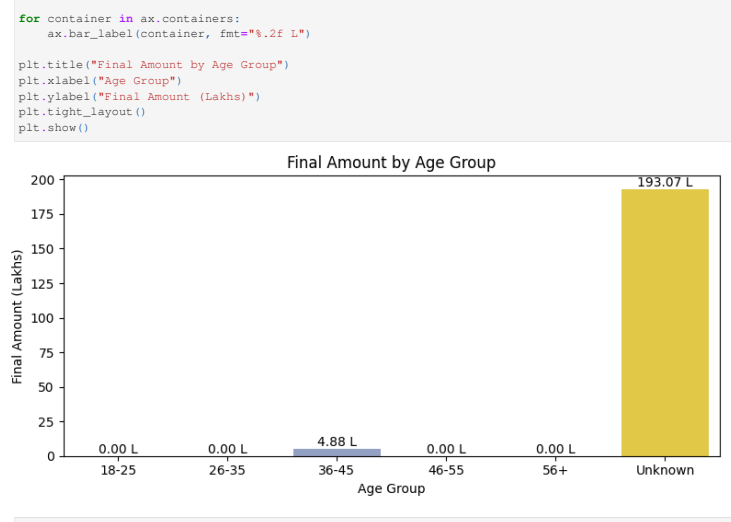
In [23]:

```python
# 5. Orders by Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

orders_by_age = df["Age Group"].value_counts().reindex(age_order, fill_value=0)

plt.figure(figsize=(8,4))
palette = sns.color_palette("Set2", n_colors=len(orders_by_age))

ax = sns.barplot(x=orders_by_age.index, y=orders_by_age.values, palette=palette)

for container in ax.containers:
    ax.bar_label(container)

plt.title("Orders by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Orders")
plt.tight_layout()
plt.show()
```



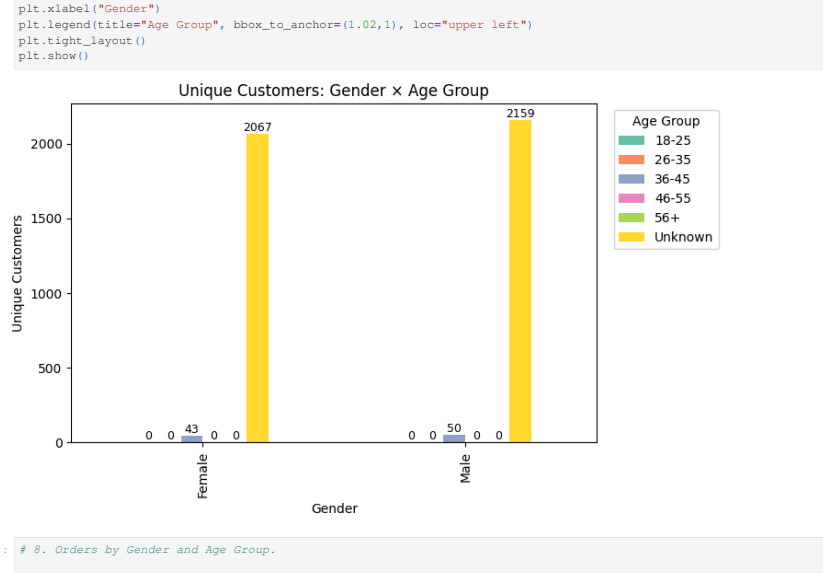
In [24]:

```python
# 6. Final amount by Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

final_by_age = (df.groupby("Age Group")["Final Amount"].sum().reindex(age_order, fill_value=0))

final_by_age_lakh = final_by_age / 1e5

plt.figure(figsize=(8,4))
palette = sns.color_palette("Set2", n_colors=len(final_by_age_lakh))

ax = sns.barplot(x=final_by_age_lakh.index, y=final_by_age_lakh.values, palette=palette)

for container in ax.containers:
    ax.bar_label(container, fmt="%.2f L")

plt.title("Final Amount by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Final Amount (Lakhs)")
plt.tight_layout()
plt.show()
```



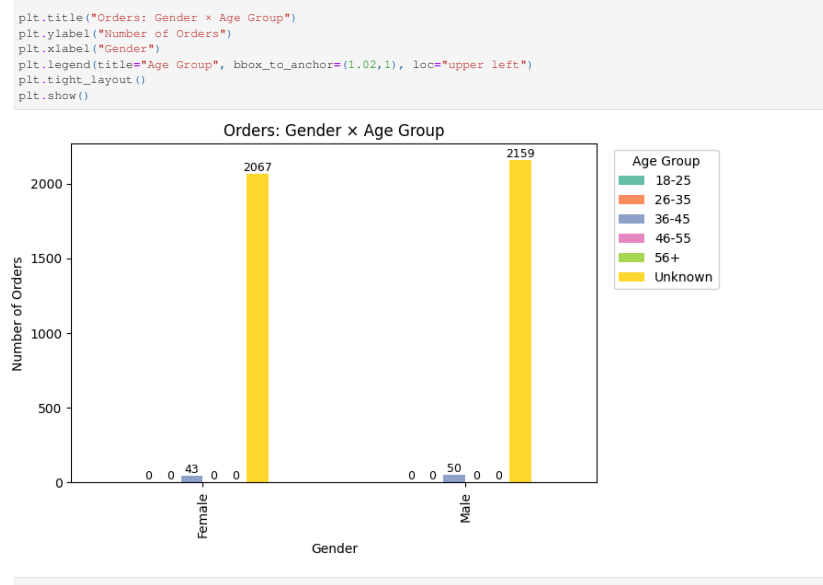
In [25]:

```python
# 7. Unique customers by Gender and Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

g_a_unique = ( unique_customers.groupby(["Gender","Age Group"])["CustomerKey"].nunique().unstack(fill_value=0).reindex(columns=age_order, fill_value=0))

ax = g_a_unique.plot(kind="bar",figsize=(8,5),color=sns.color_palette("Set2", n_colors=len(age_order)))

for container in ax.containers:
    ax.bar_label(container, fontsize=9)

plt.title("Unique Customers: Gender × Age Group")
plt.ylabel("Unique Customers")
plt.xlabel("Gender")
plt.legend(title="Age Group", bbox_to_anchor=(1.02,1), loc="upper left")
plt.tight_layout()
plt.show()
```



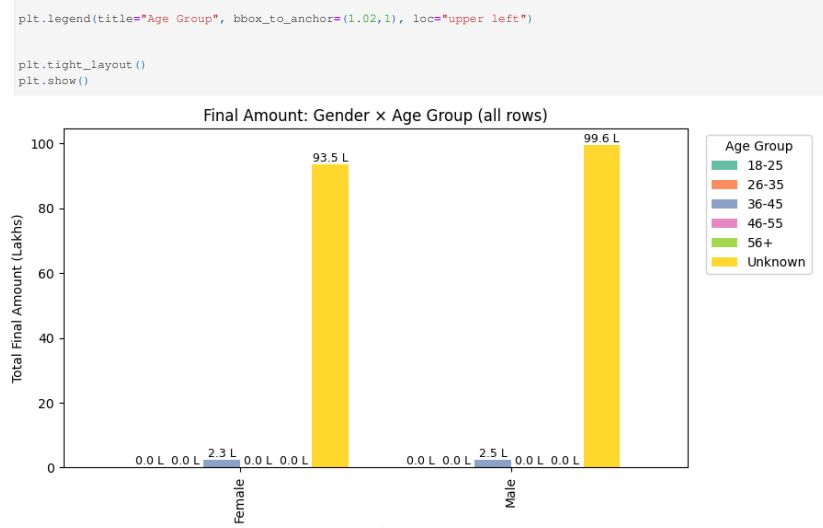
In [26]:

```python
# 8. Orders by Gender and Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

g_a_orders = (df.groupby(["Gender","Age Group"]).size().unstack(fill_value=0).reindex(columns=age_order, fill_value=0)
              .reindex(columns=age_order, fill_value=0))

ax = g_a_orders.plot(kind="bar",figsize=(8,5),color=sns.color_palette("Set2", n_colors=len(age_order)))

for container in ax.containers:
    ax.bar_label(container, fontsize=9)

plt.title("Orders: Gender × Age Group")
plt.ylabel("Number of Orders")
plt.xlabel("Gender")
plt.legend(title="Age Group", bbox_to_anchor=(1.02,1), loc="upper left")
plt.tight_layout()
plt.show()
```



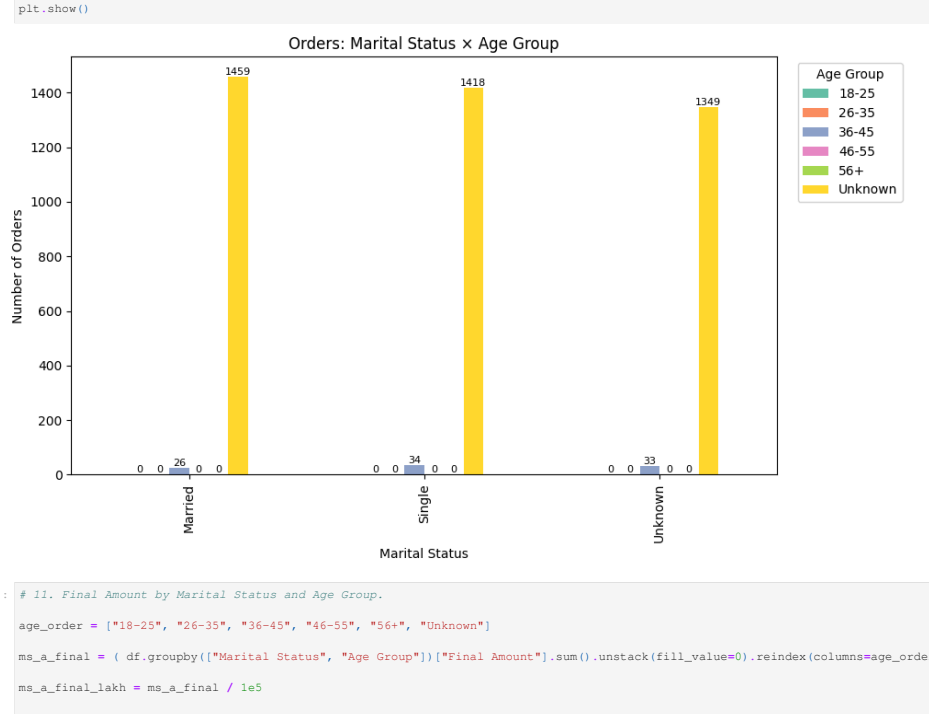
In [27]:

```python
# 9. Final amount by Gender and Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

g_a_final = (df.groupby(["Gender","Age Group"])["Final Amount"].sum().unstack(fill_value=0).reindex(columns=age_order, fill_value=0))

g_a_final_lakh = g_a_final / 1e5

ax = g_a_final_lakh.plot(kind="bar",figsize=(8,5),width=0.8,color=sns.color_palette("Set2", n_colors=len(age_order)))

for container in ax.containers:
    ax.bar_label(container, fmt="%.1f L", fontsize=8)

plt.title("Final Amount: Gender × Age Group (all rows)")
plt.ylabel("Total Final Amount (Lakhs)")
plt.xlabel("Gender")

plt.legend(title="Age Group", bbox_to_anchor=(1.02,1), loc="upper left")


plt.tight_layout()
plt.show()
```



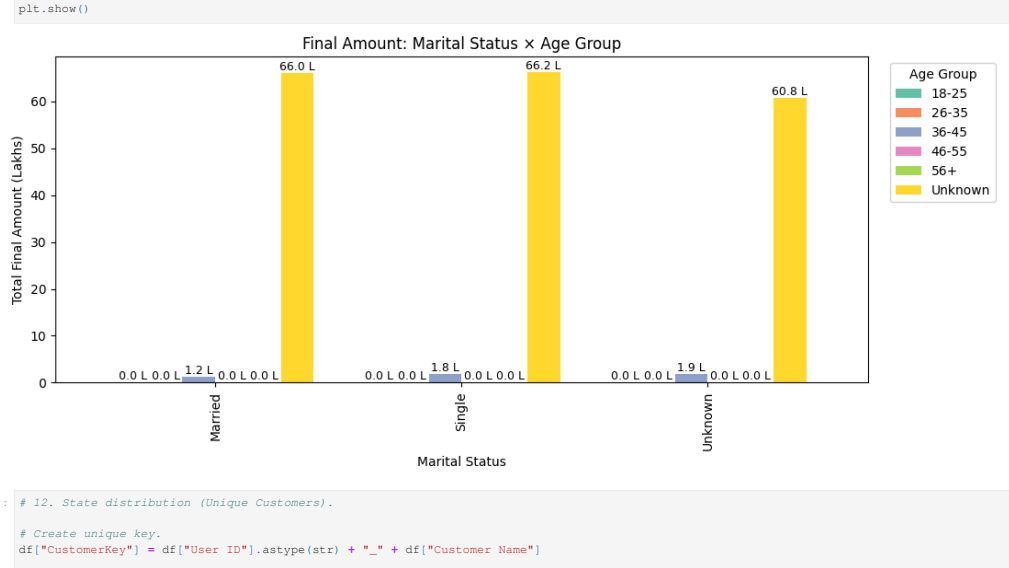
In [28]:

```python
# 10. Orders by Marital Status and Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

ms_age_orders = (df.groupby(["Marital Status", "Age Group"]).size().unstack(fill_value=0).reindex(columns=age_order, fill_value=0))

ax = ms_age_orders.plot(kind="bar", figsize=(10,6),color=sns.color_palette("Set2", n_colors=len(age_order)))

for container in ax.containers:
    ax.bar_label(container, fontsize=8)

plt.title("Orders: Marital Status × Age Group")
plt.ylabel("Number of Orders")
plt.xlabel("Marital Status")
plt.legend(title="Age Group", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



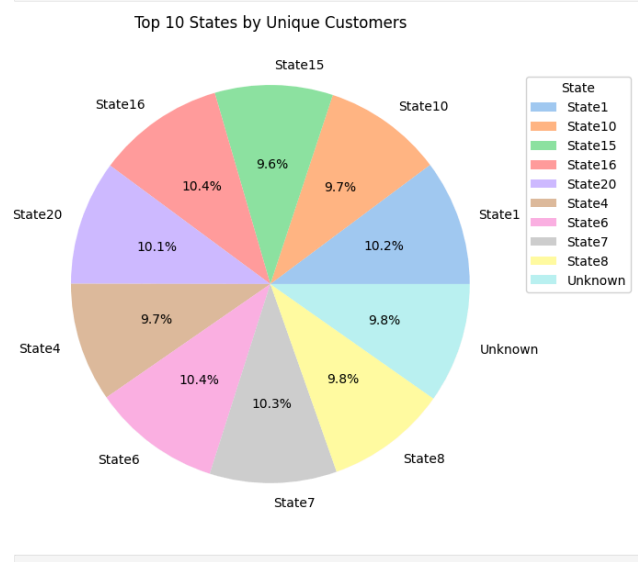
In [29]:

```python
# 11. Final Amount by Marital Status and Age Group.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

ms_a_final = ( df.groupby(["Marital Status", "Age Group"])["Final Amount"].sum().unstack(fill_value=0).reindex(columns=age_order, fill_value=0))

ms_a_final_lakh = ms_a_final / 1e5

ax = ms_a_final_lakh.plot(kind="bar", figsize=(11,5), width=0.8,color=sns.color_palette("Set2", n_colors=len(age_order)))

for container in ax.containers:
    ax.bar_label(container, fmt="%.1f L", fontsize=8)

plt.title("Final Amount: Marital Status × Age Group")
plt.ylabel("Total Final Amount (Lakhs)")
plt.xlabel("Marital Status")
plt.legend(title="Age Group", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



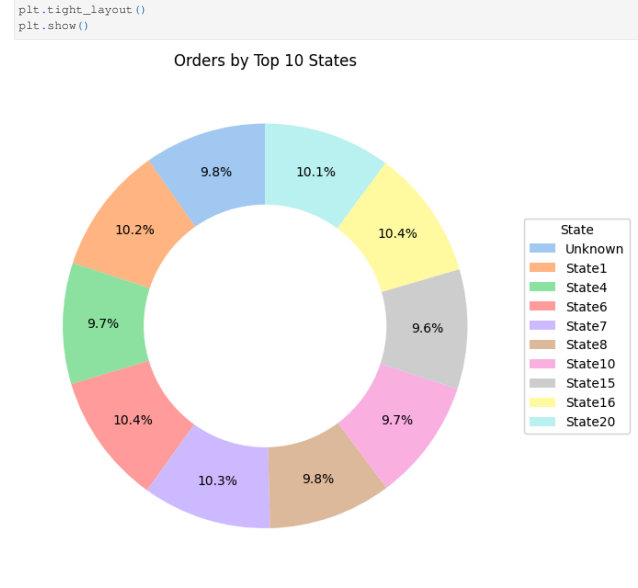
In [30]:

```python
# 12. State distribution (Unique Customers).

# Create unique key.
df["CustomerKey"] = df["User ID"].astype(str) + "_" + df["Customer Name"]

# Unique customers.
unique_customers = df.drop_duplicates(subset="CustomerKey")

# Get top 10 states by customer count.
top_states = unique_customers["State"].value_counts().nlargest(10)

top_states = top_states.sort_index()

plt.figure(figsize=(7,7))
plt.pie(top_states, labels=top_states.index, autopct="%1.1f%%",colors=sns.color_palette("pastel", n_colors=len(top_states)))

plt.title("Top 10 States by Unique Customers")

plt.legend(top_states.index, title="State", loc="center left", bbox_to_anchor=(1, 0.7))

plt.tight_layout()
plt.show()
```



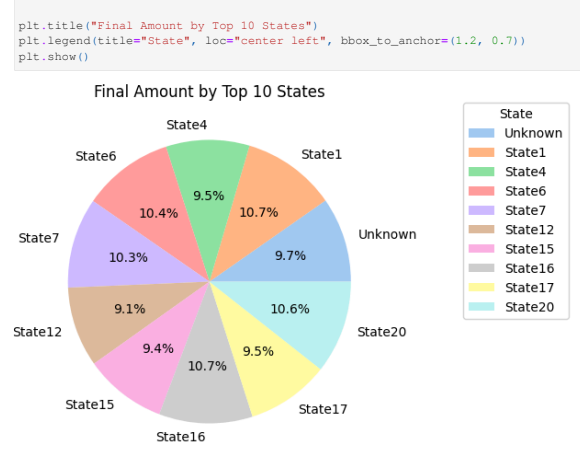
In [31]:

```python
# 13. Orders by State (Top 10).

top_states_orders = df["State"].value_counts().nlargest(10)

def state_sort_key(state):
    digits = ''.join([c for c in state if c.isdigit()])
    return int(digits) if digits else 0

top_states_orders = top_states_orders.sort_index(key=lambda x: [state_sort_key(s) for s in x])

plt.figure(figsize=(7,7))
plt.pie(top_states_orders, labels=None, autopct="%1.1f%%", wedgeprops={"width": 0.4}, startangle=90, pctdistance=0.8,
colors=sns.color_palette("pastel", n_colors=len(top_states_orders)))

plt.title("Orders by Top 10 States")
plt.legend(top_states_orders.index, title="State", loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout()
plt.show()
```



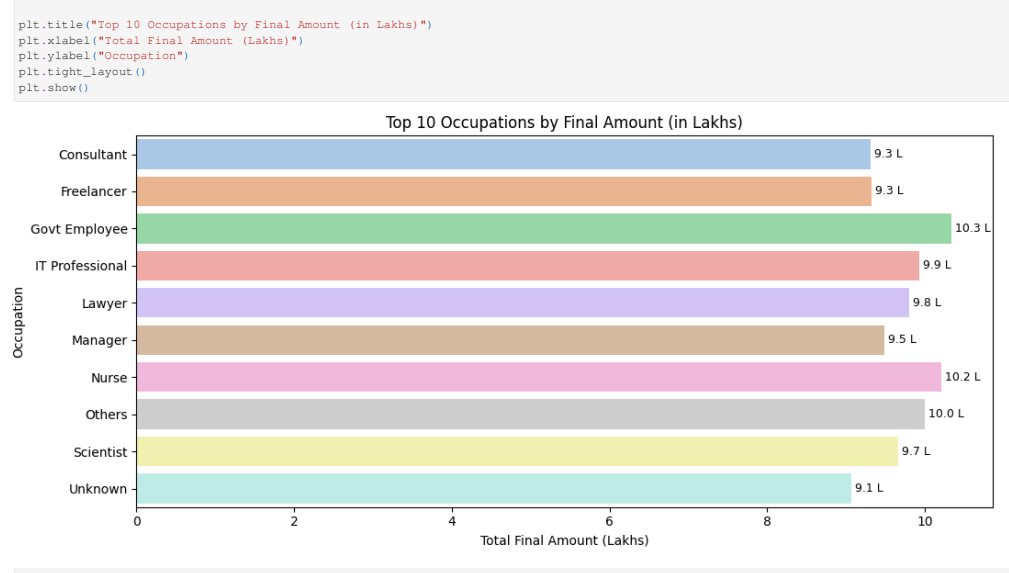
In [32]:

```python
# 14. Final amount by top 10 states.

final_by_state = df.groupby("State")["Final Amount"].sum().nlargest(10)

def state_sort_key(state):
    digits = ''.join([c for c in state if c.isdigit()])
    return int(digits) if digits else 0

final_by_state = final_by_state.sort_index(key=lambda x: [state_sort_key(s) for s in x])

plt.figure(figsize=(5,5))
plt.pie(final_by_state, labels=final_by_state.index, autopct="%1.1f%%",colors=sns.color_palette("pastel", n_colors=len(final_by_state)))

plt.title("Final Amount by Top 10 States")
plt.legend(title="State", loc="center left", bbox_to_anchor=(1.2, 0.7))
plt.show()
```



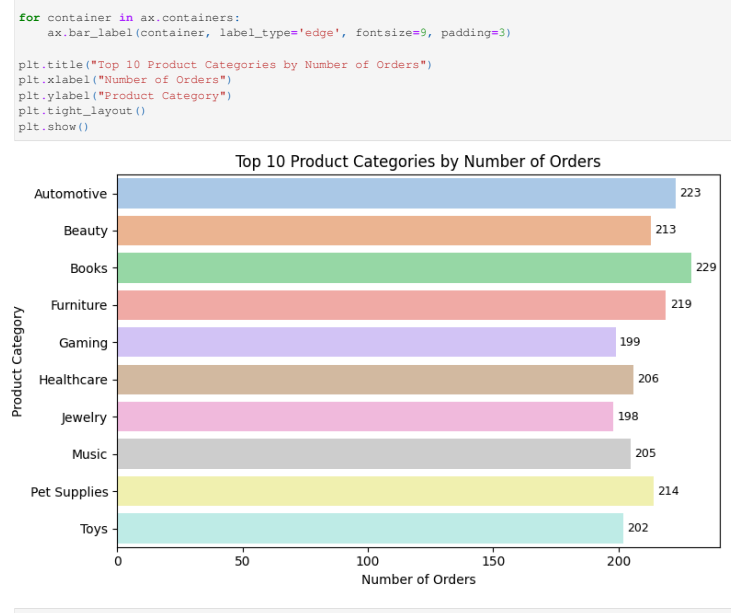
In [33]:

```python
# 15. Final amount by top 10 occupations.

final_by_occupation = df.groupby("Occupation")["Final Amount"].sum().nlargest(10)

final_by_occupation_lakh = final_by_occupation / 1e5

final_by_occupation_lakh = final_by_occupation_lakh.sort_index()

plt.figure(figsize=(11,5))
ax = sns.barplot(x=final_by_occupation_lakh.values, y=final_by_occupation_lakh.index, palette="pastel")

for container in ax.containers:
    ax.bar_label(container, label_type='edge', fontsize=9, padding=3, fmt="%.1f L")

plt.title("Top 10 Occupations by Final Amount (in Lakhs)")
plt.xlabel("Total Final Amount (Lakhs)")
plt.ylabel("Occupation")
plt.tight_layout()
plt.show()
```



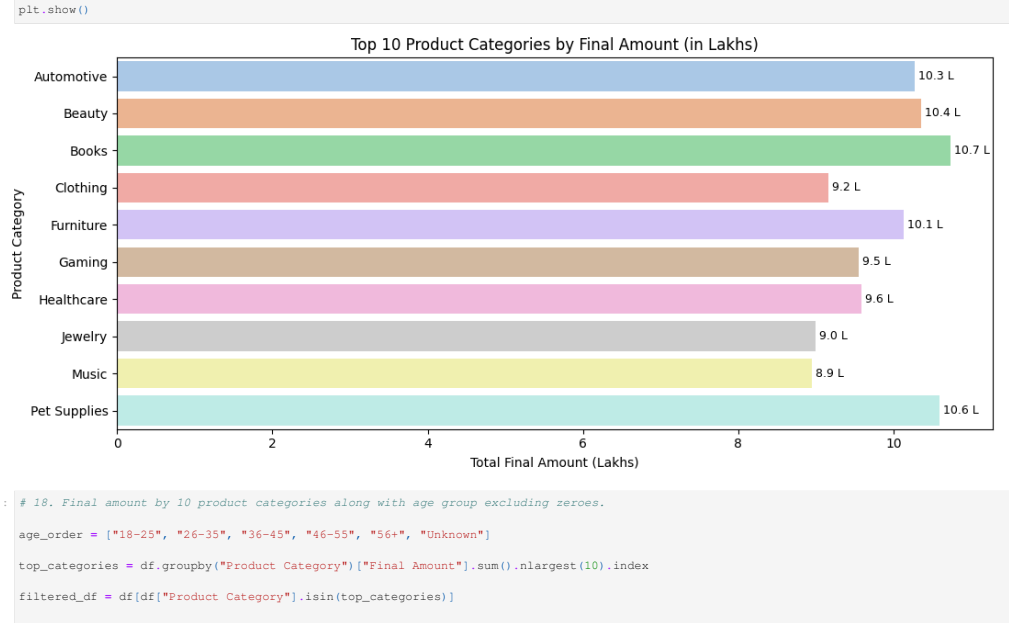
In [34]:

```python
# 16. Count of orders by product category (top 10).

category_counts = df["Product Category"].value_counts().nlargest(10)


category_counts = category_counts.sort_index()


plt.figure(figsize=(8,5))
ax = sns.barplot(x=category_counts.values, y=category_counts.index, palette="pastel")

for container in ax.containers:
    ax.bar_label(container, label_type='edge', fontsize=9, padding=3)

plt.title("Top 10 Product Categories by Number of Orders")
plt.xlabel("Number of Orders")
plt.ylabel("Product Category")
plt.tight_layout()
plt.show()
```



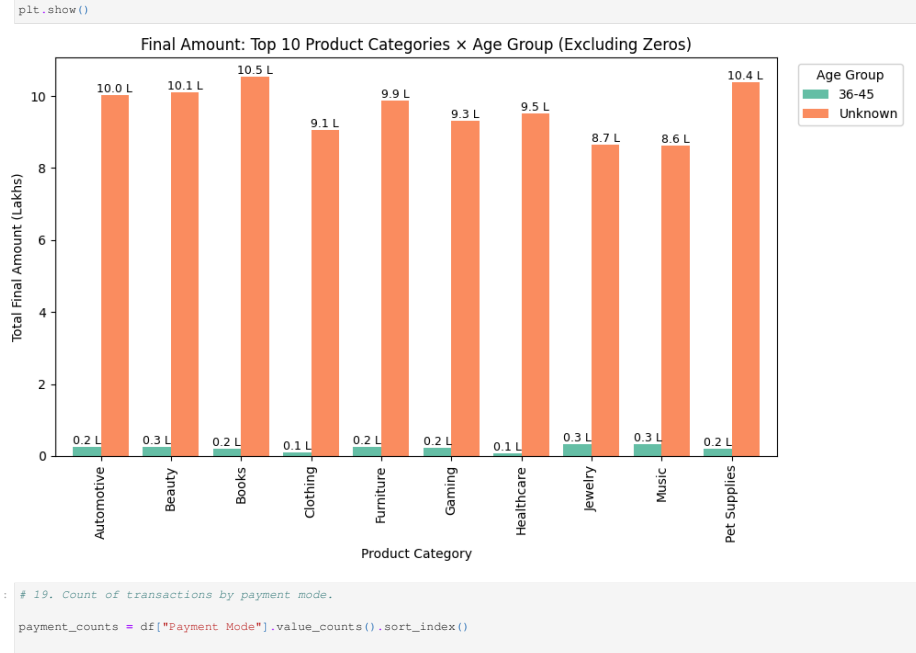
In [35]:

```python
# 17. Final amount by top 10 product categories.

final_by_category = df.groupby("Product Category")["Final Amount"].sum().nlargest(10)


final_by_category_lakh = final_by_category / 1e5


final_by_category_lakh = final_by_category_lakh.sort_index()

# Bar chart
plt.figure(figsize=(11,5))
ax = sns.barplot(x=final_by_category_lakh.values, y=final_by_category_lakh.index, palette="pastel")


for container in ax.containers:
    ax.bar_label(container, label_type='edge', fontsize=9, padding=3, fmt="%.1f L")

plt.title("Top 10 Product Categories by Final Amount (in Lakhs)")
plt.xlabel("Total Final Amount (Lakhs)")
plt.ylabel("Product Category")
plt.tight_layout()
plt.show()
```



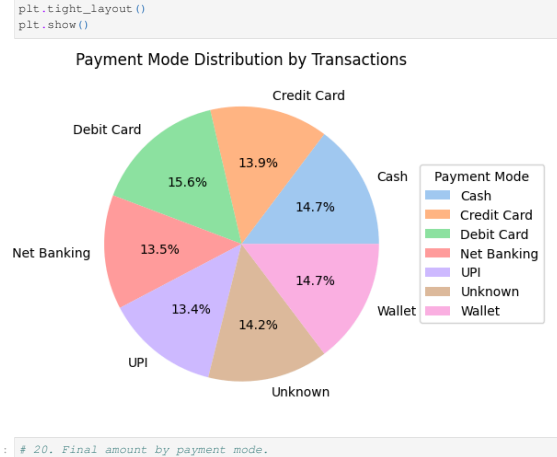
In [36]:

```python
# 18. Final amount by 10 product categories along with age group excluding zeroes.

age_order = ["18-25", "26-35", "36-45", "46-55", "56+", "Unknown"]

top_categories = df.groupby("Product Category")["Final Amount"].sum().nlargest(10).index

filtered_df = df[df["Product Category"].isin(top_categories)]

cat_age_final = (filtered_df.groupby(["Product Category", "Age Group"])["Final Amount"]
                 .sum().unstack(fill_value=0).reindex(columns=age_order, fill_value=0))

cat_age_final = cat_age_final.loc[:, (cat_age_final != 0).any(axis=0)]

cat_age_final_lakh = cat_age_final / 1e5

ax = cat_age_final_lakh.plot(kind="bar", figsize=(10,6), width=0.8,
                             color=sns.color_palette("Set2", n_colors=len(cat_age_final_lakh.columns)))

for container in ax.containers:
    ax.bar_label(container, fmt="%.1f L", fontsize=8)

plt.title("Final Amount: Top 10 Product Categories × Age Group (Excluding Zeros)")
plt.ylabel("Total Final Amount (Lakhs)")
plt.xlabel("Product Category")
plt.legend(title="Age Group", bbox_to_anchor=(1.02,1), loc="upper left")
plt.tight_layout()
plt.show()
```



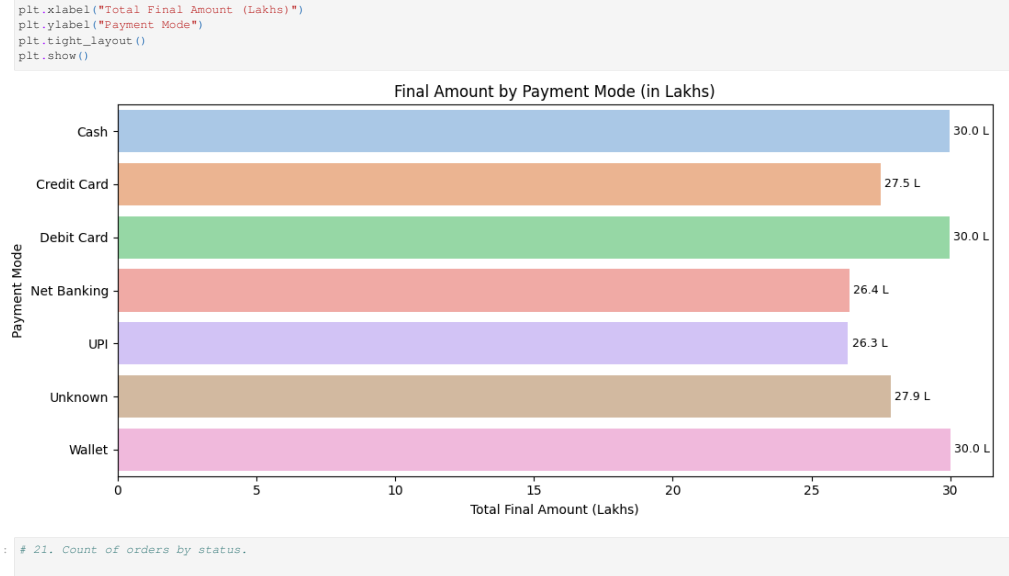
In [37]:

```python
# 19. Count of transactions by payment mode.

payment_counts = df["Payment Mode"].value_counts().sort_index()

plt.figure(figsize=(6,6))
plt.pie(payment_counts, labels=payment_counts.index, autopct="%1.1f%%",
        colors=sns.color_palette("pastel", n_colors=len(payment_counts)))

plt.title("Payment Mode Distribution by Transactions")
plt.legend(title="Payment Mode", loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout()
plt.show()
```



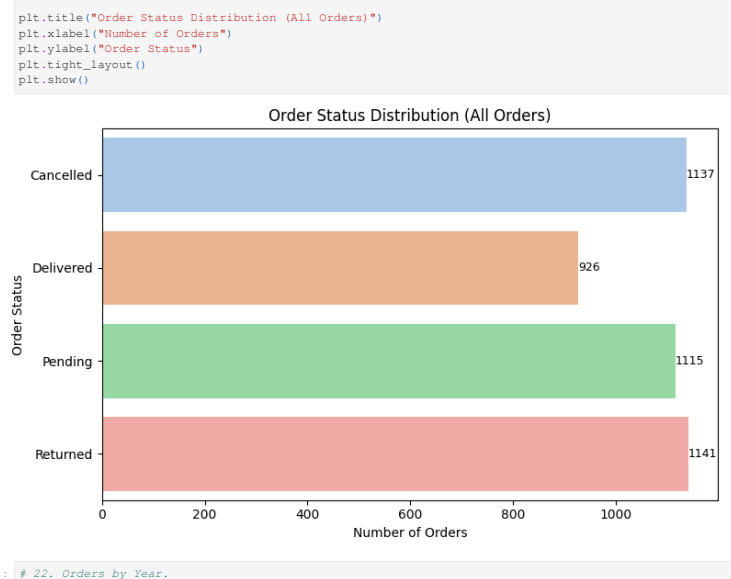
In [38]:

```python
# 20. Final amount by payment mode.

final_by_payment = df.groupby("Payment Mode")["Final Amount"].sum()

final_by_payment_lakh = final_by_payment / 1e5

final_by_payment_lakh = final_by_payment_lakh.sort_index()

plt.figure(figsize=(11,5))
ax = sns.barplot(x=final_by_payment_lakh.values, y=final_by_payment_lakh.index, palette="pastel")

for container in ax.containers:
    ax.bar_label(container, label_type='edge', fontsize=9, padding=3, fmt="%.1f L")

plt.title("Final Amount by Payment Mode (in Lakhs)")
plt.xlabel("Total Final Amount (Lakhs)")
plt.ylabel("Payment Mode")
plt.tight_layout()
plt.show()
```



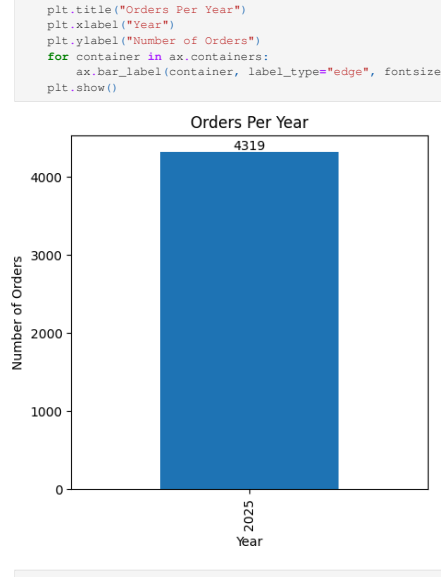
In [39]:

```python
# 21. Count of orders by status.

status_counts = df["Order Status"].value_counts().sort_index()

plt.figure(figsize=(8,5))
ax = sns.barplot(x=status_counts.values, y=status_counts.index, palette="pastel")

for container in ax.containers:
    ax.bar_label(container, label_type='edge', fontsize=9)

plt.title("Order Status Distribution (All Orders)")
plt.xlabel("Number of Orders")
plt.ylabel("Order Status")
plt.tight_layout()
plt.show()
```



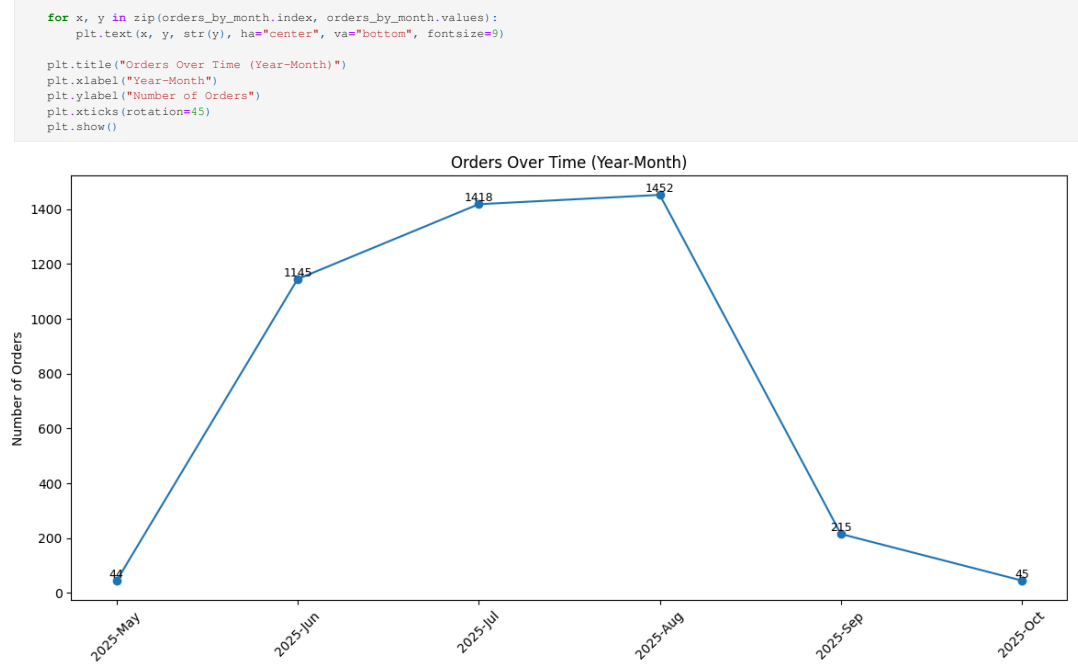
In [40]:

```python
# 22. Orders by Year.

if "Order Date" in df.columns:
    df["Year"] = df["Order Date"].dt.year  # extract year
    orders_by_year = df.groupby("Year")["User ID"].count()

    plt.figure(figsize=(5,5))
    ax = orders_by_year.plot(kind="bar")
    plt.title("Orders Per Year")
    plt.xlabel("Year")
    plt.ylabel("Number of Orders")
    for container in ax.containers:
        ax.bar_label(container, label_type="edge", fontsize=10)
    plt.show()
```



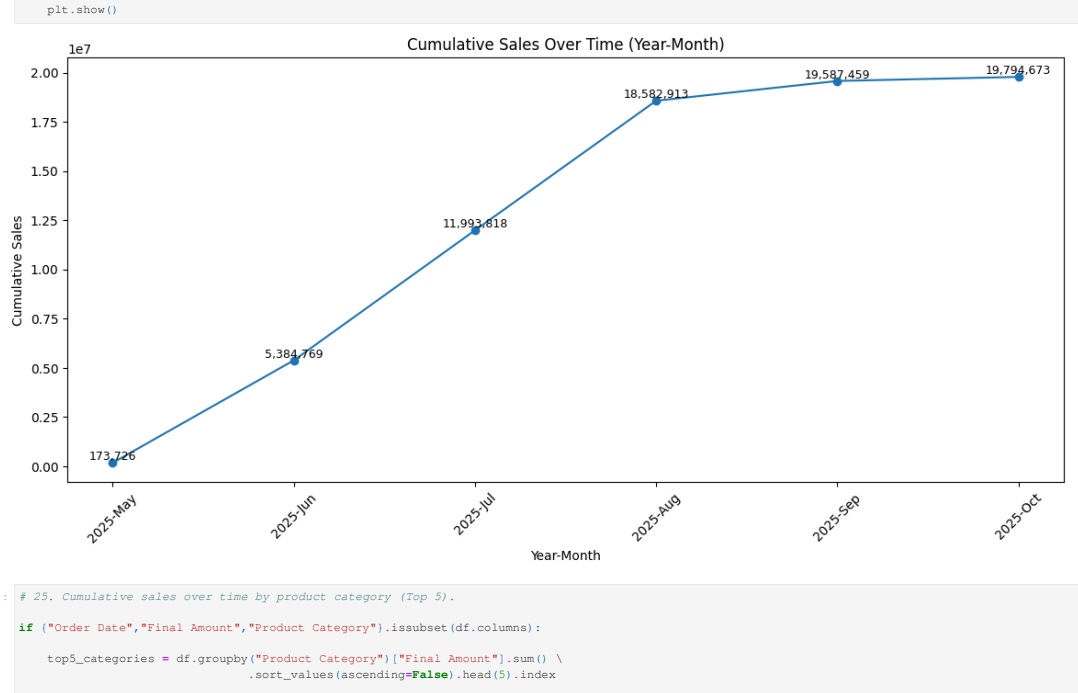
In [41]:

```python
# 23. Orders Over Time (Year-Month).

if "Order Date" in df.columns:
    # Extract Year-Month in calendar order, format as "YYYY-MMM"
    orders_by_month = df.groupby(df["Order Date"].dt.to_period("M"))["User ID"].count()
    orders_by_month.index = orders_by_month.index.to_timestamp().strftime("%Y-%b")

    plt.figure(figsize=(14,6))
    plt.plot(orders_by_month.index, orders_by_month.values, marker="o")

    for x, y in zip(orders_by_month.index, orders_by_month.values):
        plt.text(x, y, str(y), ha="center", va="bottom", fontsize=9)

    plt.title("Orders Over Time (Year-Month)")
    plt.xlabel("Year-Month")
    plt.ylabel("Number of Orders")
    plt.xticks(rotation=45)
    plt.show()
```



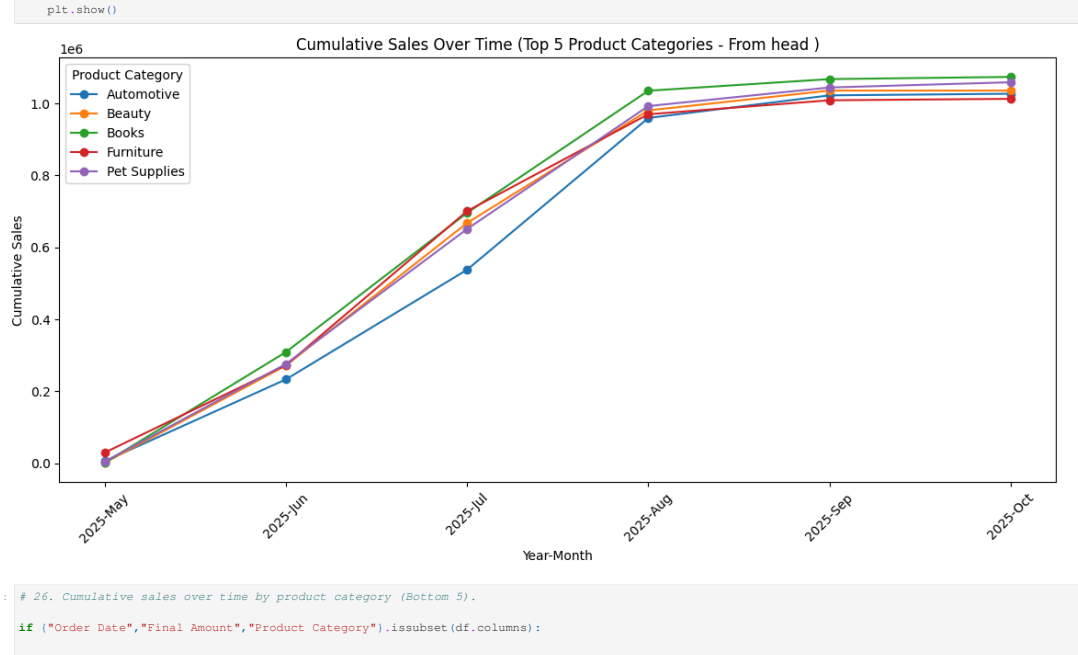
In [42]:

```python
# 24. Cumulative sales over time (Year-Month).

if {"Order Date","Final Amount"}.issubset(df.columns):

    monthly_sales = df.groupby(df["Order Date"].dt.to_period("M"))["Final Amount"].sum()

    monthly_sales.index = monthly_sales.index.to_timestamp().strftime("%Y-%b")

    cumulative_sales = monthly_sales.cumsum()

    plt.figure(figsize=(14,6))
    plt.plot(cumulative_sales.index, cumulative_sales.values, marker="o")

    for x, y in zip(cumulative_sales.index, cumulative_sales.values):
        plt.text(x, y, f"{y:,.0f}", ha="center", va="bottom", fontsize=9)

    plt.title("Cumulative Sales Over Time (Year-Month)")
    plt.xlabel("Year-Month")
    plt.ylabel("Cumulative Sales")
    plt.xticks(rotation=45)
    plt.show()
```



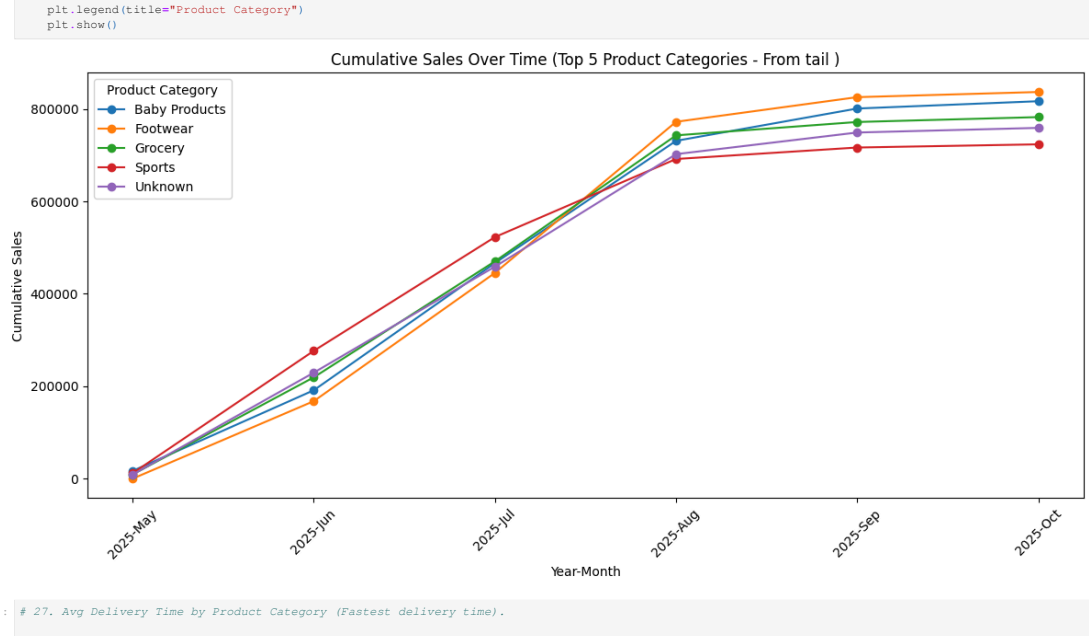
In [43]:

```python
# 25. Cumulative sales over time by product category (Top 5).

if {"Order Date","Final Amount","Product Category"}.issubset(df.columns):

    top5_categories = df.groupby("Product Category")["Final Amount"].sum() \
                        .sort_values(ascending=False).head(5).index

    df_top5 = df[df["Product Category"].isin(top5_categories)]

    monthly_sales = df_top5.groupby([df_top5["Order Date"].dt.to_period("M"),
                                     "Product Category"])["Final Amount"].sum()

    monthly_sales = monthly_sales.unstack(fill_value=0)

    cumulative_sales = monthly_sales.cumsum()

    cumulative_sales.index = cumulative_sales.index.to_timestamp().strftime("%Y-%b")

    plt.figure(figsize=(14,6))
    for col in cumulative_sales.columns:
        plt.plot(cumulative_sales.index, cumulative_sales[col], marker="o", label=col)

    plt.title("Cumulative Sales Over Time (Top 5 Product Categories - From head )")
    plt.xlabel("Year-Month")
    plt.ylabel("Cumulative Sales")
    plt.xticks(rotation=45)
    plt.legend(title="Product Category")
    plt.show()
```



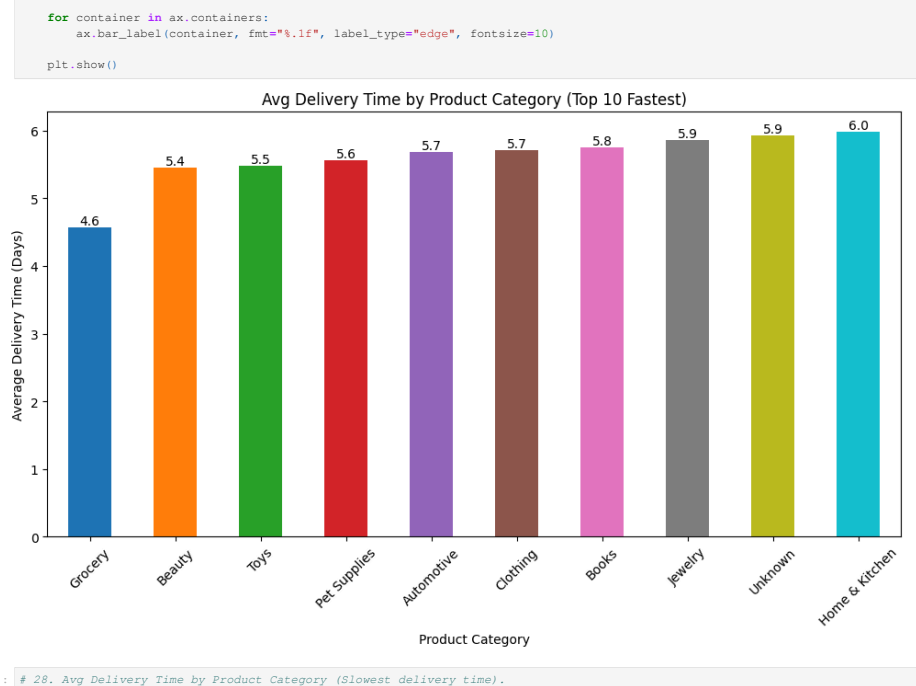
In [44]:

```python
# 26. Cumulative sales over time by product category (Bottom 5).

if {"Order Date","Final Amount","Product Category"}.issubset(df.columns):

    top5_categories = df.groupby("Product Category")["Final Amount"].sum() \
                        .sort_values(ascending=False).tail(5).index

    df_top5 = df[df["Product Category"].isin(top5_categories)]

    monthly_sales = df_top5.groupby([df_top5["Order Date"].dt.to_period("M"),
                                     "Product Category"])["Final Amount"].sum()

    monthly_sales = monthly_sales.unstack(fill_value=0)

    cumulative_sales = monthly_sales.cumsum()

    cumulative_sales.index = cumulative_sales.index.to_timestamp().strftime("%Y-%b")

    plt.figure(figsize=(14,6))
    for col in cumulative_sales.columns:
        plt.plot(cumulative_sales.index, cumulative_sales[col], marker="o", label=col)

    plt.title("Cumulative Sales Over Time (Top 5 Product Categories - From tail )")
    plt.xlabel("Year-Month")
    plt.ylabel("Cumulative Sales")
    plt.xticks(rotation=45)
    plt.legend(title="Product Category")
    plt.show()
```



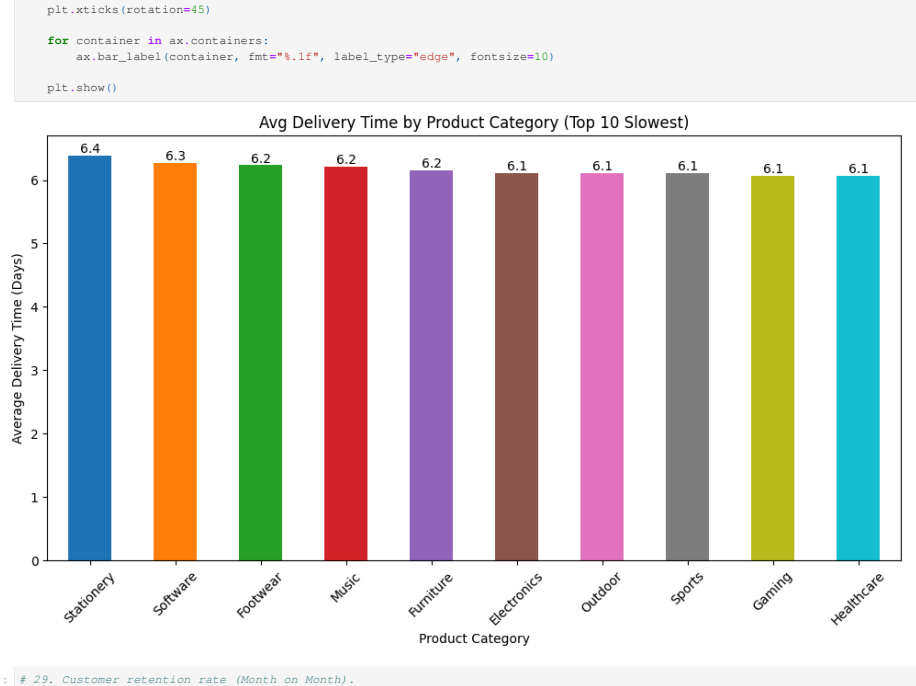
In [45]:

```python
# 27. Avg Delivery Time by Product Category (Fastest delivery time).

if {"Product Category","DeliveryTime"}.issubset(df.columns):
    avg_delivery = (
        df.groupby("Product Category")["DeliveryTime"]
          .mean()
          .sort_values(ascending=True)
          .head(10)
    )

    plt.figure(figsize=(12,6))
    ax = avg_delivery.plot(kind="bar", color=plt.cm.tab10.colors)  # different colors

    plt.title("Avg Delivery Time by Product Category (Top 10 Fastest)")
    plt.xlabel("Product Category")
    plt.ylabel("Average Delivery Time (Days)")
    plt.xticks(rotation=45)

    for container in ax.containers:
        ax.bar_label(container, fmt="%.1f", label_type="edge", fontsize=10)

    plt.show()
```

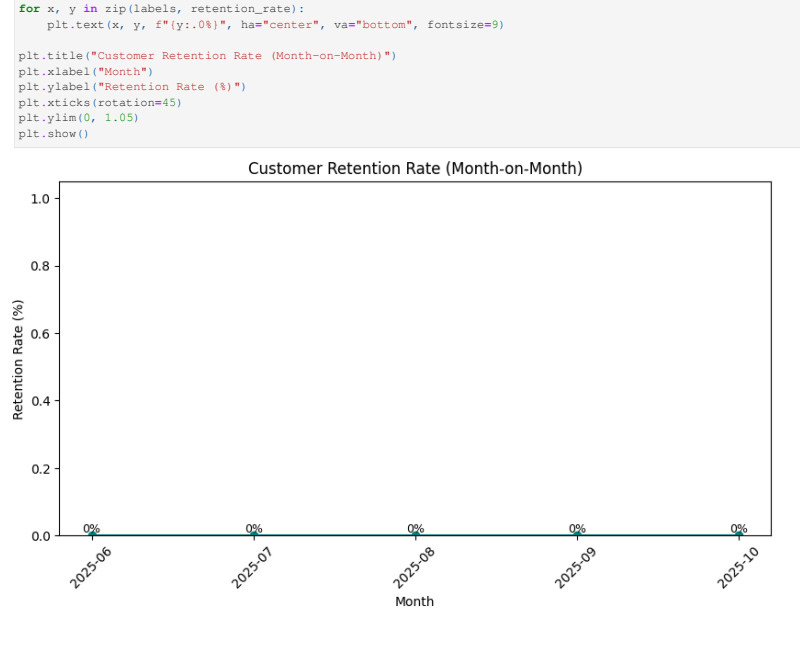


In [46]:

```python
# 28. Avg Delivery Time by Product Category (Slowest delivery time).
if {"Product Category","DeliveryTime"}.issubset(df.columns):
    avg_delivery = (
        df.groupby("Product Category")["DeliveryTime"]
          .mean()
          .sort_values(ascending=False)
          .head(10)
    )

    plt.figure(figsize=(12,6))
    ax = avg_delivery.plot(kind="bar", color=plt.cm.tab10.colors)  # different colors

    plt.title("Avg Delivery Time by Product Category (Top 10 Slowest)")
    plt.xlabel("Product Category")
    plt.ylabel("Average Delivery Time (Days)")
    plt.xticks(rotation=45)

    for container in ax.containers:
        ax.bar_label(container, fmt="%.1f", label_type="edge", fontsize=10)

    plt.show()
```



In [47]:

```python
# 29. Customer retention rate (Month on Month).

df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")

df["YearMonth"] = df["Order Date"].dt.to_period("M")

customers_per_month = df.groupby("YearMonth")["User ID"].unique()

retention_rate = [
    len(set(curr) & set(prev)) / len(set(prev)) if len(prev) > 0 else 0
    for prev, curr in zip(customers_per_month[:-1], customers_per_month[1:])
]


labels = customers_per_month.index[1:].astype(str)

plt.figure(figsize=(10,5))
plt.plot(labels, retention_rate, marker="o", color="teal")
for x, y in zip(labels, retention_rate):
    plt.text(x, y, f"{y:.0%}", ha="center", va="bottom", fontsize=9)

plt.title("Customer Retention Rate (Month-on-Month)")
plt.xlabel("Month")
plt.ylabel("Retention Rate (%)")
plt.xticks(rotation=45)
plt.ylim(0, 1.05)
plt.show()
```